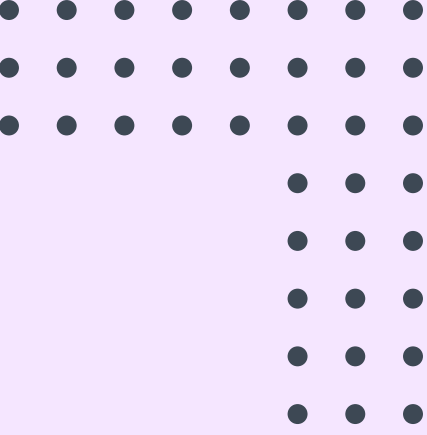
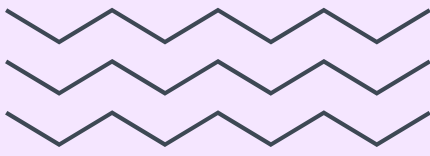


Системне програмне забезпечення

АДРЕСАЦІЯ ЗА БАЗОЮ

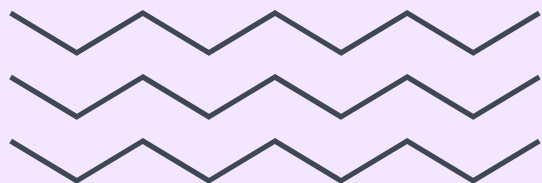
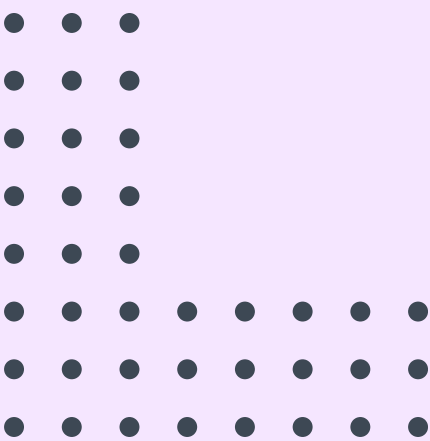
ІП-33 Соколов Олександр

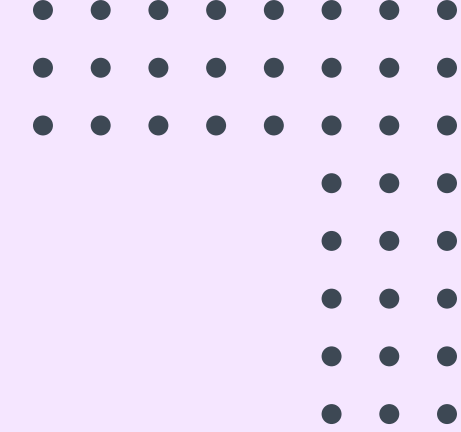


АНАЛОГІЯ

Уявімо склад з комірками

- У вас є полиця з ящиками, кожен ящик має номер.
- Уявіть, що ви знаєте тільки початкове місце першого ящика (база).
- Далі ви можете звертатися до інших ящиків, додаючи зміщення відносно цієї бази.
- База = початковий ящик, зміщення = конкретний ящик, який вам потрібен.

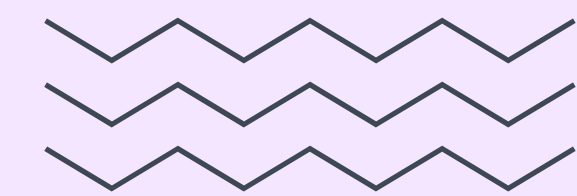
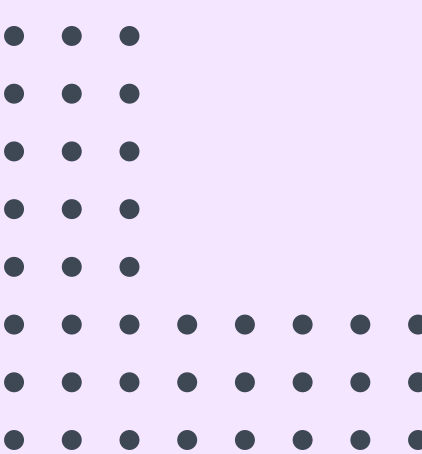


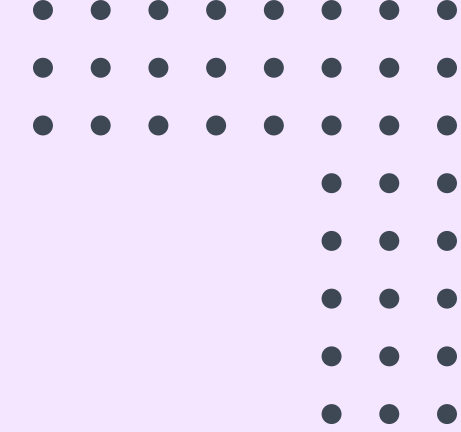


СУТЬ АДРЕСАЦІЇ ЗА БАЗОЮ

Адресація за базою (Base Addressing) є одним із способів звернення до пам'яті. Вона використовується для доступу до даних у масивах, структурах, а також для роботи зі стеком. У цьому методі фізична адреса обчислюється як значення базового регістра плюс зміщення.

Цей спосіб особливо корисний у випадках, коли необхідно зчитувати або змінювати дані за певним зміщенням від базової адреси.





ПРИНЦИП РОБОТИ

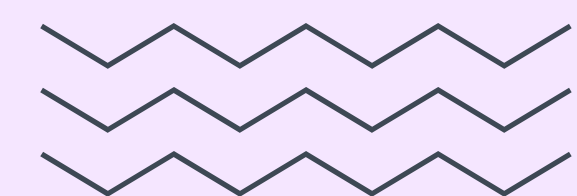
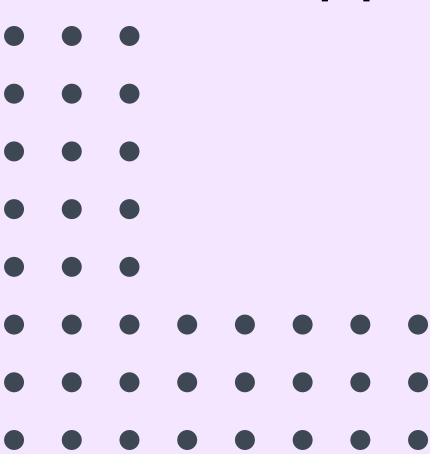
У процесорі доступ до пам'яті здійснюється за допомогою 16-бітових регістрів. У разі адресації за базою використовуються такі регістри:

- BX (Base Register) – використовується для доступу до даних у сегменті даних (DS).
- BP (Base Pointer) – зазвичай застосовується для роботи зі стеком у сегменті стеку (SS).

Формула обчислення адреси:

$$\underline{\text{Фізична адреса} = \text{базовий регістр} + \text{зміщення}}$$

Де зміщення може бути позитивним або негативним значенням, вираженим у байтах.



СИНТАКСИС

Адресація за базою використовується у командах звернення до пам'яті.



Base addressing

```
MOV AX, [BX]      ; Завантажити в AX слово за адресою BX
MOV AL, [BP]      ; Завантажити в AL байт за адресою BP
MOV AX, [BX+2]    ; Завантажити в AX слово на 2 байти далі від адреси BX
MOV DX, [BP-4]    ; Завантажити в DX слово на 4 байти раніше від BP
```


СИНТАКСИС

- Важливо зазначити, що при використанні BP як базового реєстра процесор автоматично звертається до сегменту стеку (SS), а при використанні BX — до сегменту даних (DS).
- Якщо потрібно змінити сегмент, необхідно явно задати його:



Base addressing

```
MOV AX, [SS:BP+4] ; Читаємо дані з сегмента стеку (SS)
MOV BX, [DS:BX-2] ; Читаємо дані з сегмента даних (DS)
```

ПРИКЛАД ВИКОРИСТАННЯ. РОБОТА З МАСИВОМ

Розглянемо ситуацію, коли потрібно отримати другий елемент масиву:

- OFFSET numbers дає початкову адресу масиву.
- BX вказує на перший елемент.
- (BX+4) означає, що ми беремо третій елемент (30), оскільки кожне слово займає 2 байти (тобто 2-й елемент numbers(1) має зміщення 2, а numbers(2) — 4).

```
Base addressing

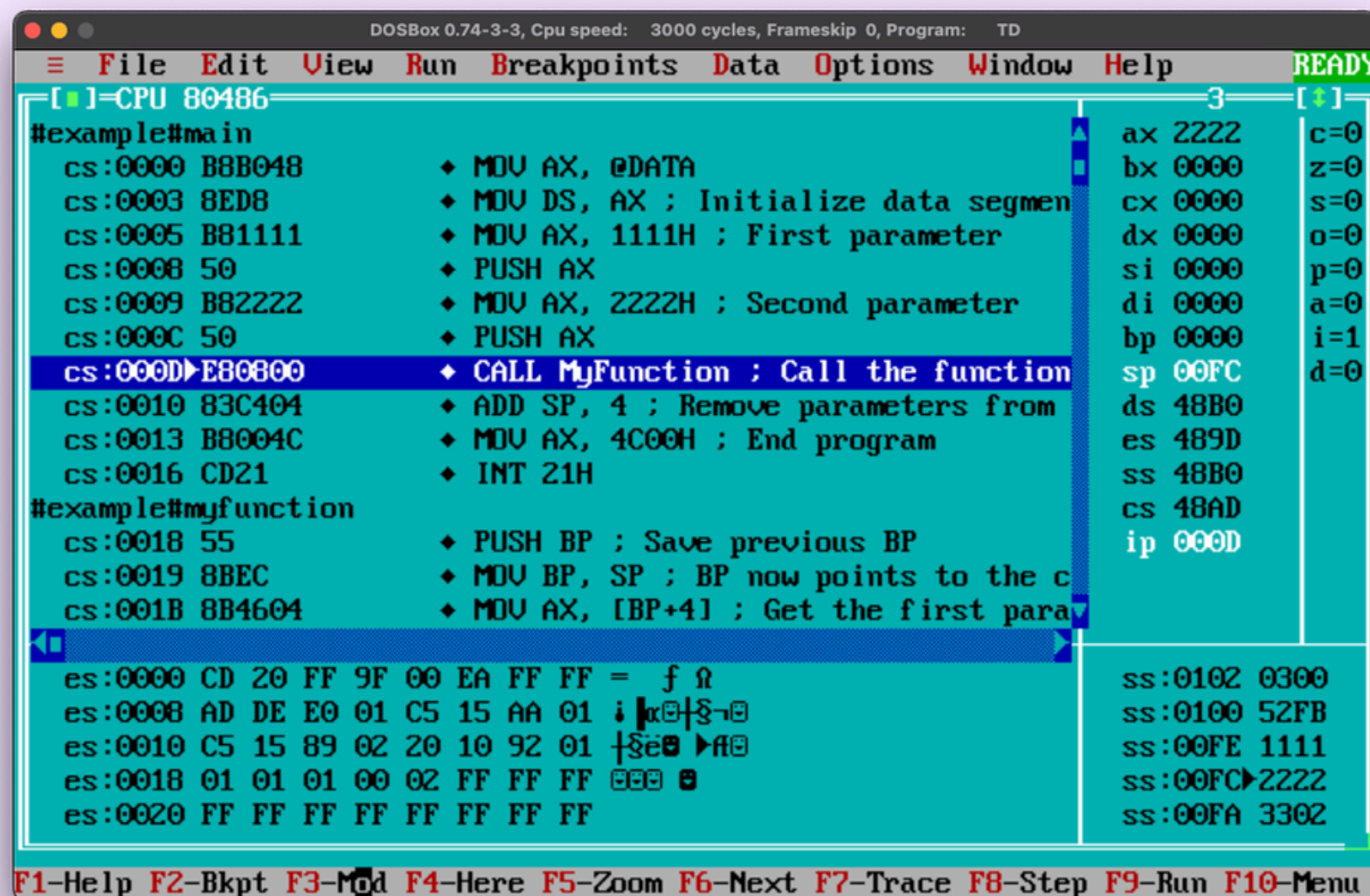
.MODEL SMALL
.STACK 100H
.DATA
    numbers DW 10, 20, 30, 40 ; Масив 16-бітних чисел
    result  DW ?             ; Змінна для результату
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX                ; Ініціалізація сегмента даних

    MOV BX, OFFSET numbers    ; BX вказує на початок масиву
    MOV AX, [BX+4]             ; Читаємо третій елемент (30)
    MOV result, AX            ; Зберігаємо у змінній result

    MOV AX, 4C00H              ; Завершення програми
    INT 21H
MAIN ENDP
END MAIN
```


ПРИКЛАД ВИКОРИСТАННЯ. РОБОТА ЗІ СТЕКОМ

До виклику функції на стеку зберігаються параметри, які будуть передані в функцію. У цьому прикладі в стеку є два параметри: 1111H та 2222H.



```
DOSBox 0.74-3-3, Cpu speed: 3000 cycles, Frameskip 0, Program: TD
File Edit View Run Breakpoints Data Options Window Help
[CPU 80486]
#example#main
cs:0000 B8B048  ♦ MOV AX, @DATA
cs:0003 8ED8      ♦ MOV DS, AX ; Initialize data segment
cs:0005 B81111    ♦ MOV AX, 1111H ; First parameter
cs:0008 50        ♦ PUSH AX
cs:0009 B82222    ♦ MOV AX, 2222H ; Second parameter
cs:000C 50        ♦ PUSH AX
cs:000D E80800    ♦ CALL MyFunction ; Call the function
cs:0010 83C404    ♦ ADD SP, 4 ; Remove parameters from stack
cs:0013 B8004C    ♦ MOV AX, 4C00H ; End program
cs:0016 CD21      ♦ INT 21H
#example#myfunction
cs:0018 55        ♦ PUSH BP ; Save previous BP
cs:0019 8BEC      ♦ MOV BP, SP ; BP now points to the current top of the stack
cs:001B 8B4604    ♦ MOV AX, [BP+4] ; Get the first parameter (1111H)
es:0000 CD 20 FF 9F 00 EA FF FF = f 0
es:0008 AD DE E0 01 C5 15 AA 01  i 0 0 0 0
es:0010 C5 15 89 02 20 10 92 01  i 0 0 0 0
es:0018 01 01 01 00 02 FF FF FF 00 0 0
es:0020 FF FF FF FF FF FF FF FF
ax 2222 c=0
bx 0000 z=0
cx 0000 s=0
dx 0000 o=0
si 0000 p=0
di 0000 a=0
bp 0000 i=1
sp 00FC d=0
ds 48B0
es 489D
ss 48B0
cs 48AD
ip 000D
ss:0102 0300
ss:0100 52FB
ss:00FE 1111
ss:00FC 2222
ss:00FA 3302
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX      ; Initialize data segment

    ; Push parameters onto the stack (simulating function call)
    MOV AX, 1111H   ; First parameter
    PUSH AX
    MOV AX, 2222H   ; Second parameter
    PUSH AX

    CALL MyFunction ; Call the function

    ADD SP, 4        ; Remove parameters from stack after the call

    MOV AX, 4C00H    ; End program
    INT 21H
MAIN ENDP

MyFunction PROC
    PUSH BP          ; Save previous BP
    MOV BP, SP        ; BP now points to the current top of the stack

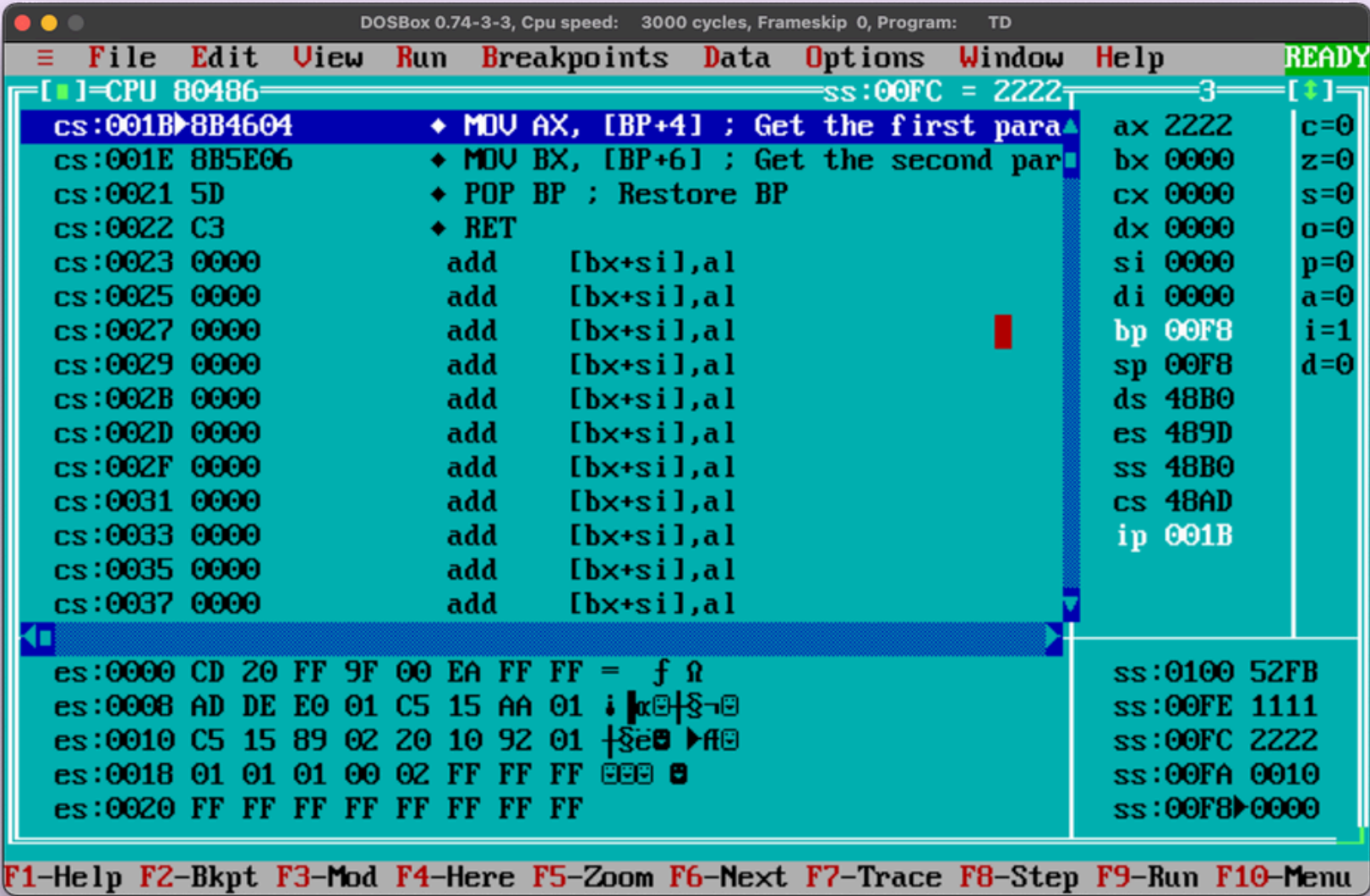
    MOV AX, [BP+4]    ; Get the first parameter (1111H)
    MOV BX, [BP+6]    ; Get the second parameter (2222H)

    POP BP            ; Restore BP
    RET
MyFunction ENDP

END MAIN
```


ПРИКЛАД ВИКОРИСТАННЯ. РОБОТА ЗІ СТЕКОМ

Після виконання інструкції MOV BP, SP регістр BP тепер вказує на поточну вершину стеку. Всі наступні параметри можуть бути отримані через BP з відповідним зміщенням.



The screenshot shows the DOSBox 0.74-3-3 interface with the CPU window open. The assembly code is as follows:

Address	Instruction	Comment
cs:001B	8B4604	MOV AX, [BP+4] ; Get the first parameter
cs:001E	8B5E06	MOV BX, [BP+6] ; Get the second parameter
cs:0021	5D	POP BP ; Restore BP
cs:0022	C3	RET
cs:0023	0000	add [bx+si], al
cs:0025	0000	add [bx+si], al
cs:0027	0000	add [bx+si], al
cs:0029	0000	add [bx+si], al
cs:002B	0000	add [bx+si], al
cs:002D	0000	add [bx+si], al
cs:002F	0000	add [bx+si], al
cs:0031	0000	add [bx+si], al
cs:0033	0000	add [bx+si], al
cs:0035	0000	add [bx+si], al
cs:0037	0000	add [bx+si], al

The registers window on the right shows the following values:

Register	Value
ax	2222
bx	0000
cx	0000
dx	0000
si	0000
di	0000
bp	00F8
sp	00F8
ds	48B0
es	489D
ss	48B0
cs	48AD
ip	001B

The memory window at the bottom shows the following values:

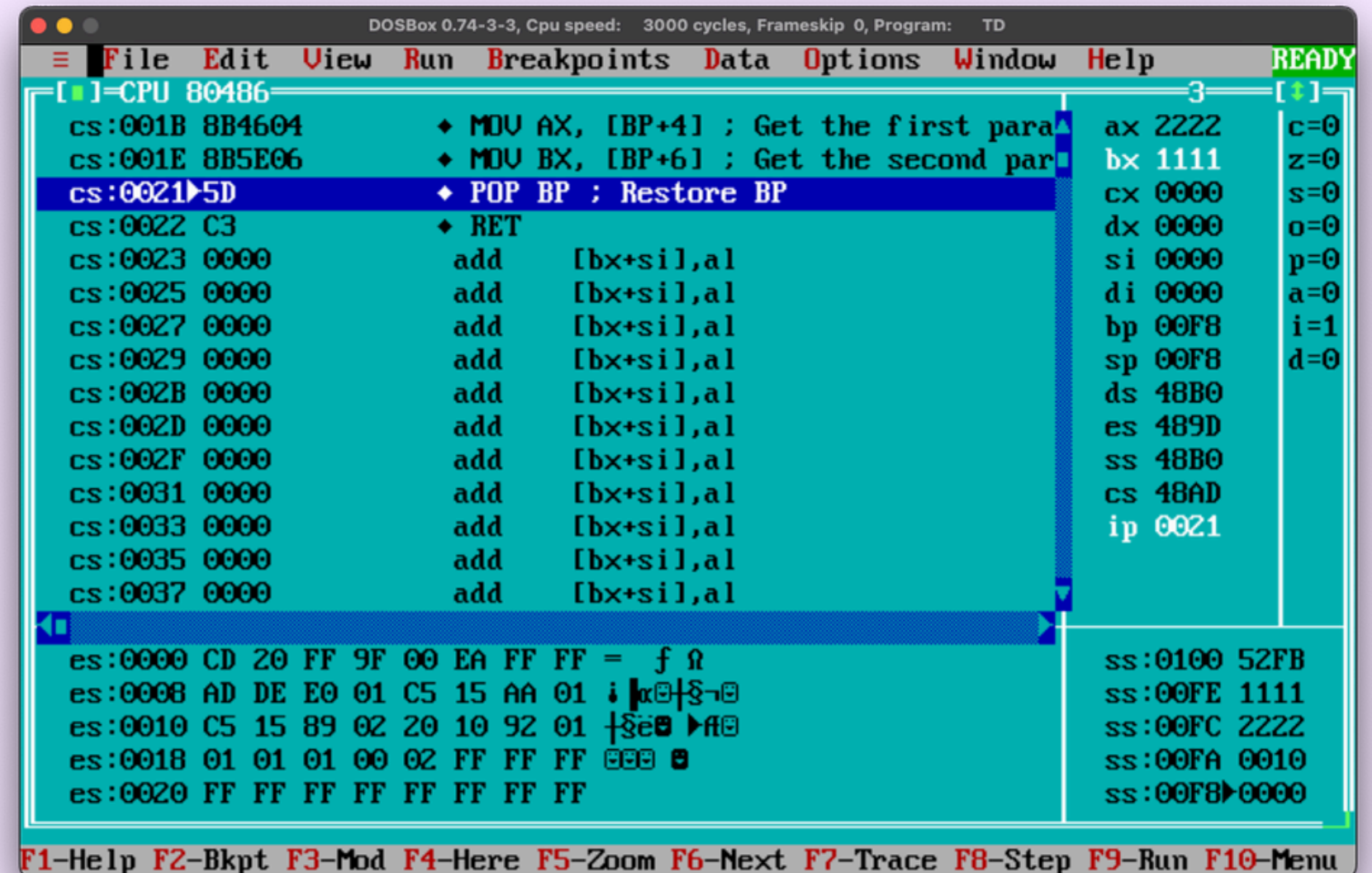
Address	Value
es:0000	CD 20 FF 9F 00 EA FF FF = f R
es:0008	AD DE E0 01 C5 15 AA 01 ; x S-
es:0010	C5 15 89 02 20 10 92 01 Se >A
es:0018	01 01 01 00 02 FF FF FF
es:0020	FF FF FF FF FF FF FF

The status bar at the bottom shows the following keyboard shortcuts:

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

ПРИКЛАД ВИКОРИСТАННЯ. РОБОТА ЗІ СТЕКОМ

За допомогою BP ми отримуємо доступ до параметрів функції. У цьому прикладі AX отримує значення першого параметра (1111H), а BX — другого параметра (2222H).



The screenshot shows the DOSBox 0.74-3-3 interface with the following assembly code and register values:

Address	Code	Comment	Register/Value
cs:001B	8B4604	♦ MOV AX, [BP+4] ; Get the first para	ax 2222
cs:001E	8B5E06	♦ MOV BX, [BP+6] ; Get the second par	bx 1111
cs:0021	5D	♦ POP BP ; Restore BP	cx 0000
cs:0022	C3	♦ RET	dx 0000
cs:0023	0000	add [bx+si],al	si 0000
cs:0025	0000	add [bx+si],al	di 0000
cs:0027	0000	add [bx+si],al	bp 00F8
cs:0029	0000	add [bx+si],al	sp 00F8
cs:002B	0000	add [bx+si],al	ds 48B0
cs:002D	0000	add [bx+si],al	es 489D
cs:002F	0000	add [bx+si],al	ss 48B0
cs:0031	0000	add [bx+si],al	cs 48AD
cs:0033	0000	add [bx+si],al	ip 0021
cs:0035	0000	add [bx+si],al	
cs:0037	0000	add [bx+si],al	

Address	Code	Comment	Register/Value
es:0000	CD 20 FF 9F 00 EA FF FF	= f R	ss:0100 52FB
es:0008	AD DE E0 01 C5 15 AA 01	i x0 S-0	ss:00FE 1111
es:0010	C5 15 89 02 20 10 92 01	+Se0 >A0	ss:00FC 2222
es:0018	01 01 01 00 02 FF FF FF	000 0	ss:00FA 0010
es:0020	FF FF FF FF FF FF FF		ss:00F8 0000

Legend: F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

ПРИКЛАД ВИКОРИСТАННЯ. РОБОТА ЗІ СТЕКОМ

Після завершення функції ми відновлюємо значення BP за допомогою інструкції POP BP, що дозволяє коректно повернутися до попереднього стану стеку.

DOSBox 0.74-3-3, Cpu speed: 3000 cycles, Frameskip 0, Program: TD

File Edit View Run Breakpoints Data Options Window Help

[]-CPU 80486

#example#main

cs:0000 B8B048 ♦ MOV AX, @DATA

cs:0003 8ED8 ♦ MOV DS, AX ; Initialize data segmen

cs:0005 B81111 ♦ MOV AX, 1111H ; First parameter

cs:0008 50 ♦ PUSH AX

cs:0009 B82222 ♦ MOV AX, 2222H ; Second parameter

cs:000C 50 ♦ PUSH AX

cs:000D E80800 ♦ CALL MyFunction ; Call the function

cs:0010 83C404 ♦ ADD SP, 4 ; Remove parameters from

cs:0013 B8004C ♦ MOV AX, 4C00H ; End program

cs:0016 CD21 ♦ INT 21H

#example#myfunction

cs:0018 55 ♦ PUSH BP ; Save previous BP

cs:0019 8BEC ♦ MOV BP, SP ; BP now points to the c

cs:001B 8B4604 ♦ MOV AX, [BP+4] ; Get the first para

ax 4C00 c=0

bx 1111 z=0

cx 0000 s=0

dx 0000 o=0

si 0000 p=1

di 0000 a=1

bp 0000 i=1

sp 0100 d=0

ds 48B0

es 489D

ss 48B0

cs 48AD

ip 0016

es:0000 CD 20 FF 9F 00 EA FF FF = f R

es:0008 AD DE E0 01 C5 15 AA 01 ; |x@|S-@

es:0010 C5 15 89 02 20 10 92 01 |Se@ >A@

es:0018 01 01 01 00 02 FF FF FF @@@ @

es:0020 FF FF FF FF FF FF FF FF

ss:0108 0004

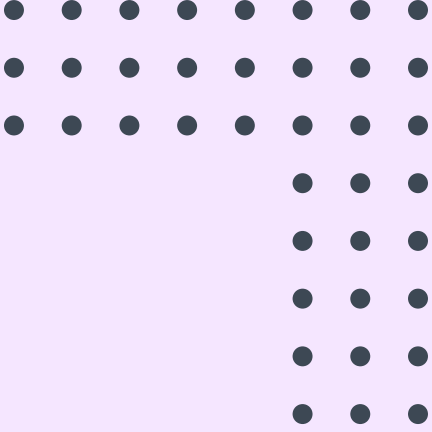
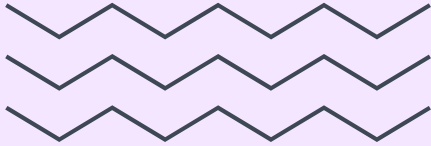
ss:0106 0000

ss:0104 0024

ss:0102 0300

ss:0100 52FB

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

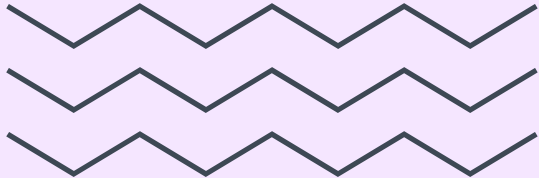


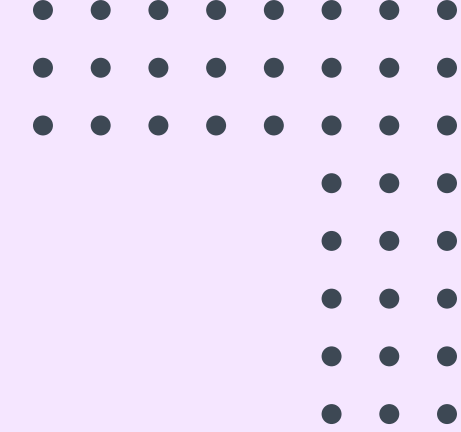
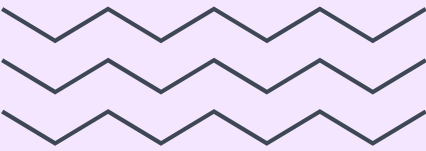
ПЕРЕВАГИ ТА НЕДОЛІКИ

ПЕРЕВАГИ

- Гнучкість: Можна працювати зі структурами даних різної складності.
- Ефективність: Дозволяє швидко отримувати доступ до пам'яті.
- Зручність у роботі зі стеком: Використовується для передачі аргументів і локальних змінних у процедурах.

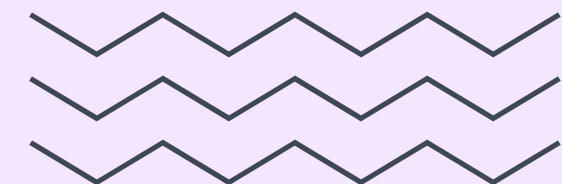
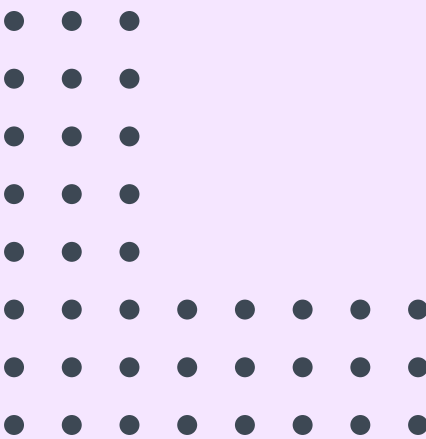
НЕДОЛІКИ

- Вимагає точного керування зміщенням – необхідно правильно визначати зміщення, щоб не допустити помилок.
 - Обмежена кількість базових регістрів – 8086 має обмежену кількість регістрів (BX, BP), що може викликати труднощі в складних програмах.
 - Не підходить для динамічного розміщення пам'яті – працює лише з жорстко заданими адресами.
- 



ВИСНОВОК

Адресація за базою є важливим механізмом, який дозволяє зручно працювати з масивами, структурами та стеком. Вона широко використовується у системному програмуванні та організації ефективного доступу до пам'яті.





Системне програмне забезпечення

ДЯКУЮ!

ІП-33 Соколов Олександр